Assignment 2: Audio Identification

Rabin Bhatta

***Abstract*—This report evaluates a rule based audio identification system for short noisy recordings. The task is to recover the exact database recording that produced each query clip. The pipeline uses audio loading and resampling, short-time Fourier transform (STFT), spectral peak picking, peak-density limiting, anchor-target landmark hashing, inverted-index database construction, and query matching by repeated offset voting. At the front end, an STFT spectrogram and a sparse constellation-style peak representation are used. At the back end, landmark hashing and offset voting follow Wang [1]. Two peak-picking routes were compared, and the maximum-filter route was kept because it gave a steadier peak density. On the 200-track classical and pop subset of GTZAN, together with 213 noisy smartphone-style queries, the final system achieves Top-1 = 0.7512, Top-3 = 0.8028, and MAP@3 = 0.7762. The correct track is returned at rank 1 for 160 queries and inside the top 3 for 171 queries. Average query time is 0.2472 s, and the fingerprint database contains 200,200 unique hash keys and 2,237,403 postings.**

I. Introduction

Audio identification is the task of taking an unknown query and finding the exact recording it came from. The goal is to find the one exact database track that matches the query. A well known example of this kind of system is Shazam, and the main reference for this report is Wang's landmark hashing method [1].

The reference database contains 200 tracks from the classical and pop subsets of GTZAN. The query set contains 213 short noisy clips recorded on smartphones in different acoustic conditions. The database audio is 22.05 kHz but the queries are 44.1 kHz. So the queries are resampled to 22.05 kHz first. If this is skipped, the spectrogram grid changes and the hashes will not match. The system returns a tab separated top-3 output file for each query.

A practical reason for keeping the system fully rule based is because a sparse STFT peak front end which was developed from the lab material is easy to implement and inspect. This is strong enough to support a searchable retrieval system. Wang's method then gives the rest of the pipeline with anchor-target landmark hashes, an inverted index, and offset voting [1]. Muller's book and the FMP materials were also used as background and as direct references for the implemented code and metrics [5], [6].

One practical use case for this type of exact audio identification system is rights monitoring. A producer may upload an instrumental online, and later a different musician may use it without permission and release it on a digital streaming platform. If access to a platform scale reference database were available. One example is a catalogue of tracks distributed on Spotify, the uploaded track could in principle be screened for possible unauthorised reuse. This will depend on database access and the uploaded signal staying close enough to the original recording for exact recording fingerprinting to work.

II. Background and Related Work

Audio fingerprinting literature can be grouped into a few broad families. The first family is spectral-peak landmark fingerprinting, second is compact sub-fingerprint methods and third is learned embeddings. This project belongs to the first family, but the others are still worth mentioning because they help explain why this design was chosen.

A. Spectral-peak landmark fingerprinting

Wang's "An Industrial-Strength Audio Search Algorithm" [1] is the most important reference for this project. The basic idea is to convert a spectrogram into a sparse set of local peaks and then pair these peaks into anchor-target landmarks. Each pair becomes a hash by using the two frequency values and the time difference between them. Then these hashes are stored in an inverted index together with the track id and anchor time. If there is a query, shared hashes will vote for a database-minus-query time offset. The correct match should produce many votes at one consistent offset, while wrong matches should produce a flatter distribution of votes. This is the main search idea used in the system.

B. Sub-fingerprint methods

A different route is given by Haitsma and Kalker [2]. Their system does not use sparse landmarks. Instead it divides the spectrum into frequency bands and forms compact bit based sub fingerprints which are then matched with Hamming distance. This route is very compact and fast, but the queries here are noisy smartphone recordings, so Wang's sparse landmark representation looked like the safer starting point. It is more selective, and it is also easier to inspect when something goes wrong.

C. Other peak-based rule systems

Dupraz and Richard [3] also proposed a frequency based fingerprinting system built from salient peaks. Their method uses peak based features together with frequency tolerant matching, candidate pre-selection, and robustness to speed changes. The system here does not directly implement their method but the paper is relevant as it shows another practical peak-based route for degraded conditions.

D. Learned fingerprinting

More recent systems often use learned embeddings rather than handmade landmarks. A neural network is trained so that noisy queries and clean references from the same recording are mapped close together. This can improve robustness but it needs training data and a more complex engineering pipeline. The system here stays with a fully rule based method because it is easier to interpret and better matched to the scope of the task.

E. Evaluation framing

The report follows the FMP material [6] for evaluation. The database contains K items, each query Q has a relevant set I_Q and the system ranks the database using a similarity function. Each query here has one correct database track and as a result the rank-based metrics become simple. Top-1 checks whether the right track is first and top-3 checks whether it appears anywhere in the top three. MAP@3 also becomes easy to interpret because with one relevant item per query it reduces to a reciprocal-rank style score [6].

III. System Overview

The implemented system is an audio identification pipeline based on spectral peaks and landmark hashes, following Wang's method. First, it builds fingerprints for the database. Then it stores them in an index and finally applies the same front-end fingerprinting process to each query before matching the query against the index. Here is the full chain:

audio → STFT → band-limited spectrogram → spectral peaks → density-limited peaks → landmark hashes → inverted index → offset voting → top-3 matches

Both database and the query tracks use the same fingerprint representation, so both sides stay on the same time-frequency grid if the hashes are going to match at all.

IV. Method

A. Audio loading and sample-rate normalisation

Each file is loaded in mono using `librosa.load(..., sr=config["sr"], mono=True)`. The working sample rate is fixed at 22050 Hz, so the 44.1 kHz query files are resampled to the same rate as the database before fingerprint extraction starts. This is important because without this step, the time-frequency grid would not match between the database and the query. In that case, the same recording could produce different spectrogram coordinates and the hash matches would fall sharply.

B. STFT representation

Here the system computes a magnitude STFT using a Hann window, window length $N = 2048$, and hop length $H = 512$. The magnitude spectrogram is first band-limited so that only frequencies between $f_{\min} = 80$ Hz and $f_{\max} = 5000$ Hz are kept. Finally the remaining bins are converted to decibels.

$$X(n,k) = \sum_m x(m+nH)w(m)e^{-j2\pi km/N}$$

$$|X(n,k)|_{\mathrm{dB}} = 20\log_{10}\left(\frac{|X(n,k)|}{\max|X|}\right)$$

At this stage the time-frequency representation used by the rest of the pipeline is produced. The lower cut removes very low-frequency rumble and the upper cut removes high-frequency regions that are often unstable in the noisy query recordings. As a result the next peak stage becomes more selective.

C. Spectral peaks and constellation map

Next the dense spectrogram is reduced to a sparse set of important points. This is done by forming a constellation-style peak map. Following the audio identification discussion in Muller [5] and the FMP material [6], the idea is to keep only local maxima that survive inside a small time-frequency neighbourhood and also exceed a magnitude floor.

In the implementation, a 2-D maximum filter is applied to the spectrogram. A point is kept if it is equal to the filtered value in its neighbourhood and if it is within `threshold_db` decibels of the global maximum. The final configuration uses `dist_freq`=8, `dist_time`=8, and `threshold_db`=30.

Peak selection matters because the peak map is much smaller than the full spectrogram, but it still keeps the events that are most useful for matching. A likely reason this works is that local peaks often survive noise and filtering better than dense magnitudes.

A second peak route based on `skimage.feature.peak_local_max` was also implemented. This route stays in the code as an alternative, but it is not the default used for the final reported results.

D. Peak-density limiting

After the raw peaks are found, the system applies a second sparsification step. Peaks are sorted by descending magnitude, grouped into one-second buckets and only the strongest `peaks_per_second` peaks are kept from each bucket. In the final configuration, `peaks_per_second`=40.

Peak-density limiting matters because loud sections of the spectrogram can otherwise dominate the fingerprint. The index gets much larger and accidental matches become more common if too many peaks are kept. If too few peaks are kept, useful landmarks are lost. The realised mean density is 26.19 peaks per second and 786.02 peaks per track, so the upper limit of 40 is not always reached in the final system.

E. Landmark hashes

At this stage the selected peaks are converted into landmark hashes using Wang's anchor-target idea [1]. Each peak can act as an anchor. Later peaks that fall inside a limited target zone are paired with it. Each pair produces a hash key and the anchor time stamp.

$$h(p_0,p_1) = \left(\left\lfloor\frac{k_0}{q_f}\right\rfloor, \left\lfloor\frac{k_1}{q_f}\right\rfloor, \left\lfloor\frac{n_1-n_0}{q_t}\right\rfloor\right)$$

$$\text{posting} = (\text{track\_id}, n_0)$$

The values for the final system are `fanout`=15, `target_time_dist`=80, `target_freq_dist`=120, $q_f = 4$, and $q_t = 2$.

That gives a hash that is specific enough to be useful, but not so strict that a small one bin difference causes a miss. This is why light quantisation is used here. This makes the hash less brittle under noisy conditions and small STFT differences.

F. Database indexing

Each database track is fingerprinted and the hashes are stored in a Python dictionary that acts as an inverted index:

$$L : h \rightarrow [(\text{track\_id}, n_0), \ldots]$$

In practice the query hashes can be looked up directly by key instead of comparing a query against every waveform in the database. The final index for the 200 tracks used here contains 200,200 unique hash keys and 2,237,403 postings. The serialised size on disk is 20.66 MiB. That shows the representation is compact enough for this scale.

G. Query matching by repeated-offset voting

A query is processed through the same front end to produce query hashes. An inverted index is checked for each query hash. Every matched posting casts one vote for a candidate time offset:

$$\Delta = n_0^D - n_0^Q$$

This means if the query really came from one database track, many matched hashes should agree on roughly the same offset. If the track is wrong then a few accidental matches may still happen. However the votes are much less likely to gather around one dominant offset.

Each candidate track is scored by the highest vote count in its offset histogram. Ties are then broken first by total vote count and then by a simple offset-based rule used in the implementation. The top three tracks are returned.

H. Required output

The system writes a tab separated output file. In that file each line contains the query filename followed by the top three predicted database filenames.

V. Engineering Exploration

This section explains the main tuning choices. These values are not meant to be universal. It only explains why they were kept for this dataset.

A. Choice of peak-picking route

Two peak-picking routes were tested. The first route used `peak_local_max` on the log spectrogram with a relative threshold. The second route used a 2-D maximum filter with an absolute decibel floor. In informal experiments, the maximum-filter route gave a more controlled peak density. A likely reason for this is that the absolute decibel floor is easier to interpret than the relative threshold and the separate time and frequency neighbourhood widths also matched the chosen STFT settings better. This route was therefore kept as the default.

B. Choice of peak density

The parameter `peaks_per_second` had a large effect on both retrieval and storage. Very high values such as 80 or more made the index much larger without clear accuracy gains. Very low values such as 10 or less removed too much useful landmark evidence, especially for the classical tracks where many harmonic peaks are present. The final choice of 40 was a practical compromise.

C. Choice of fan-out and target zone

Fan-out and target-zone parameters control how many hashes are produced per anchor. Smaller values reduced index size, but they also reduced repeated offset evidence for noisy queries. Larger values increased the index without giving a clear benefit. The final choice, $F = 15$ with `target_time_dist`=80 and `target_freq_dist`=120, worked quite well for this dataset.

D. Choice of hash quantisation

Quantisation values $q_f = 4$ and $q_t = 2$ were chosen so that small frame-level differences would not turn otherwise-correct hashes into misses. When $q_f = 1$ and $q_t = 1$ were used, performance dropped because too many hashes failed to match exactly. On the other hand very large quantisation values made false matches more likely because the hash alphabet became too small. The final values were therefore chosen empirically.

VI. Evaluation Methodology

The evaluation uses 200 database tracks and 213 noisy queries. Each query has one intended matching track and the ground truth can be read from the filename. For example, `classical.00000-snippet-10-0.wav` corresponds to `classical.00000.wav`.

The report focuses on rank-based metrics because the returned list length is fixed at three. Top-1 accuracy is the fraction of queries where the correct track is first. Top-3 accuracy is the fraction where the correct track appears anywhere in the top three. MAP@3 is computed by giving a score of 1, $\frac{1}{2}$, $\frac{1}{3}$, or 0 depending on whether the correct track appears at rank 1, 2, 3, or not in the top three at all. Because there is one relevant item per query, MAP@3 is numerically equivalent here to a truncated reciprocal-rank score [6].

Precision-recall curves and the maximal F-measure were considered, but they were not reported. In this setting they do not add much beyond Top-1, Top-3, and MAP@3, because there is only one relevant item per query and the returned list length is fixed at three.

VII. Results

A. Main retrieval results

These two tables show the same result from slightly different angles. The system is correct at rank 1 for about three quarters of the queries. Top-3 only adds eleven more successful cases. So the main problem is not small reranking errors. The harder problem is that some queries miss the correct track entirely.

TABLE I
MAIN RETRIEVAL RESULTS

Metric	Result	Count interpretation
Top-1 accuracy	0.7512	160 / 213 correct at rank 1
Top-3 accuracy	0.8028	171 / 213 correct in top 3
MAP@3 (= MRR@3)	0.7762	Mean reciprocal-rank style score

TABLE II
RANK DISTRIBUTION OF THE CORRECT TRACK

Outcome	Count	Percentage
Correct at rank 1	160	75.12%
Correct at rank 2	10	4.69%
Correct at rank 3	1	0.47%
Not in top 3	42	19.72%

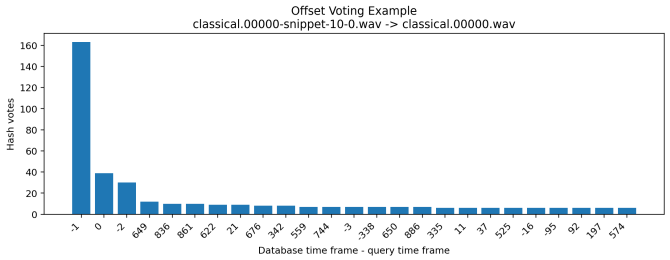

Fig. 1. Offset voting histogram for a correctly retrieved query.

B. Comparison to literature

A direct numerical comparison with Wang [1] is not simple. Wang used a much larger database and mostly cleaner queries, while the system here uses 200 tracks and noisy smartphone-style excerpts. So the absolute numbers should be interpreted with caution. What can still be compared is the general behaviour. The system shows the same repeated-offset behaviour for correct matches, and it also shows the same kind of failure where noisy or ambiguous landmarks lead to complete misses rather than just a small change in rank.

C. Implementation statistics

The build stage is a one off cost of 39.30 s, which is about 0.197 s per track. The average query time is 0.2472 s, so the system still behaves like an interactive search system at this dataset scale. The database file is 20.66 MiB for 200 tracks, which is about 106 KiB per track. That is small enough to support the idea that Wang-style landmark systems are compact as well as searchable at this scale.

D. Offset voting example

Figure 1 is useful because it shows the actual decision rule. The dominant offset bin is near -1 and this bin is clearly larger than the neighbouring bins. This means many shared hashes agree on the same query-to-database time shift. This is exactly the kind of behaviour predicted by Wang's offset voting argument [1].

E. Good and bad retrieval case studies

As you can see in Figure 2 it shows the successful case is `classical.00000-snippet-10 -0.wav` matched to `classical.00000.wav`. The bad case is `classical.00003-snippet-10-0.wav`, where the wrong top-1 prediction is `classical.00081.wav`.

TABLE III
IMPLEMENTATION STATISTICS

Statistic	Value
Database tracks indexed	200
Query clips processed	213
Fingerprint build time	39.30 s
Fingerprint database size on disk	20.66 MiB
Unique hash keys in inverted index	200,200
Total postings in inverted index	2,237,403
Average retained peaks per track	786.02
Average retained peaks per second	26.19
Total query-set processing time	52.64 s
Average query time	0.2472 s
Fastest query	0.0838 s
Slowest query	0.6618 s

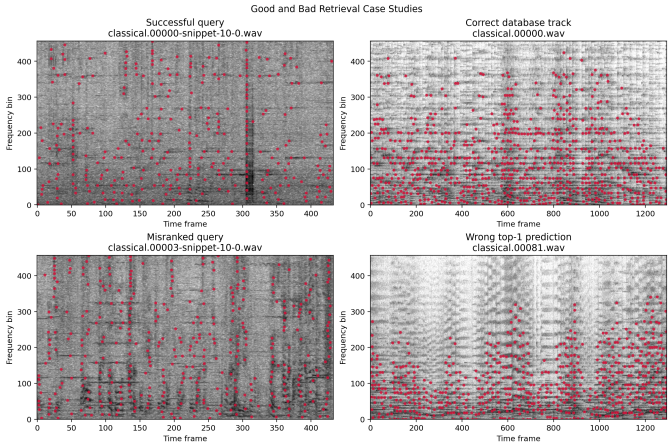

Fig. 2. Good and bad retrieval case studies.

The noisy query still keeps a relatively structured set of low- and mid-frequency landmarks in the successful example. The correct database track contains similar repeated activity, so many anchor-target hashes can still agree on one offset. This leads to a clear rank-1 match.

In the bad example, the query still has many peaks but the pattern looks more ambiguous. The wrong candidate also contains repeated low-frequency and harmonic ladders. This makes accidental but still plausible landmark matches more likely. The figure does not prove one single reason, but it does show the broader failure mode in a clear way.

VIII. DISCUSSION

A. Why the method works when it works

Three parts do most of the work here. First the STFT peak map turns a dense spectrogram into a sparse and more stable representation. Second, landmark hashes are more specific than isolated peaks because they encode a relation between two events rather than one event alone. Finally the offset voting acts like a coincidence detector. This lets true matches stand out even when some spurious matches are present. The results back this up quite clearly.

B. Why the method fails when it fails

Those 42 missed queries can be thought of in two groups. In the first group, the noisy front end simply does not preserve enough strong and distinctive peaks. In the second group a wrong database track still shares enough similar peak pairs with the query to build a false dominant offset. The bad classical example in Figure 2 fits this second case quite well.

C. Why Top-3 still matters

Top-3 matters because the system returns three ranked results. A system that often places the correct track at rank 2 can still be useful. However in the results only eleven extra cases are recovered between Top-1 and Top-3. This shows that the main weakness is not reranking. The main weakness is recall under noisy conditions.

D. How the results compare to the literature

The results are weaker compared with a commercial systems, with a miss rate of 19.72%. Even so, the general behaviour is still consistent with Wang [1]. The repeated-offset pattern is there and the main failure mode is still confusion or loss of distinctive landmark evidence. This is also in line with the audio identification discussion in Muller [5] and the FMP material [6].

IX. Limitations and Future Work

The system has some clear limitations. First it targets exact recording identification only. It does not try to handle pitch shift, time-scale modification, or version-level similarity. Second, the chosen parameters are tuned to this dataset and should not be treated as universal settings. Third, the method depends heavily on the quality of the preserved peaks which is why some failures are complete misses rather than small ranking errors. Fourth, the code still keeps `peak_local_max` as an alternative route, which means `scikit-image` remains in the dependency list even though the maximum-filter route is the default.

There are also several clear next steps. Panako-style triple hashes [4] would be a natural extension if pitch and time-scale robustness are needed. This is relevant in current online use because recordings are often reposted in slowed-down or sped-up form via UGC content from listeners and users in the market. Also the current parameters could be explored more systematically, especially `peaks_per_second`, `fanout`, `target_time_dist`, q_f, and q_t. To add to that, this Wang-style system could be compared more directly to a fuller implementation of Dupraz and Richard's method [3]. Finally, a learned embedding system could be tested as a stronger but more complex alternative.

X. Reproducibility

For reproducibility, the implementation mainly uses two Python files: `main.py` for fingerprinting and matching, and `eval.py` for metrics, plots, and runtime checks. The file `output.txt` stores each query filename followed by the top three predicted tracks, and the main settings are centralised in `DEFAULT_CONFIG`.

XI. Conclusion

This report has presented a rule based audio identification system for short noisy query recordings, using an STFT front end, sparse constellation-style peaks, Wang-style landmark hashes, an inverted index, and repeated offset voting. The final results are Top-1 = 0.7512, Top-3 = 0.8028, and MAP@3 = 0.7762, with 160 of 213 queries correct at rank 1 and 171 of 213 correct inside the top 3. The average query time is 0.2472 s, and the index contains 200,200 unique hash keys and 2,237,403 postings. The main weakness is the 42 complete misses, so future work should mainly target landmark stability under noise. These results show that the main design choices were reasonable for this dataset, even though the miss rate is still high enough to leave clear room for improvement.

References

[1] A. L.-C. Wang, "An Industrial-Strength Audio Search Algorithm," in *Proc. 4th Int. Conf. Music Information Retrieval (ISMIR)*, 2003.

[2] J. Haitsma and T. Kalker, "A Highly Robust Audio Fingerprinting System," in *Proc. 3rd Int. Conf. Music Information Retrieval (ISMIR)*, 2002.

[3] E. Dupraz and G. Richard, "Robust Frequency-Based Audio Fingerprinting," in *Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP)*, 2010.

[4] J. Six and M. Leman, "Panako: A Scalable Acoustic Fingerprinting System Handling Time-Scale and Pitch Modification," in *Proc. 15th Int. Soc. Music Information Retrieval Conf. (ISMIR)*, 2014, pp. 259–264.

[5] M. Muller, *Fundamentals of Music Processing: Audio, Analysis, Algorithms, Applications*. Cham, Switzerland: Springer, 2015.

[6] M. Muller, *FMP Notebooks*, AudioLabs Erlangen. Available: https://www.audiolabs-erlangen.de/resources/MIR/FMP/